OPTION PROFILING and Built-in Optimisations

PicoMite MMBasic - Performance Optimisation Guide

Overview

OPTION PROFILING counts how often each command and each subroutine or function runs, and times the subroutines and functions, so that you can see where a program spends its time. The rest of this document describes the optimisations the interpreter applies to every program by itself, with nothing to switch on, so that you know which forms of a statement are already fast.

Earlier versions also had a statement cache, OPTION TRACECACHE, with OPTION CACHE DEBUG and OPTION CACHE SUB. It was removed in V6.04.00: it rarely paid for itself and could make programs slower. Its one generally useful part, the DO loop fast path, is kept and is now in every build (see below). A program that still uses one of the removed OPTIONs stops at that line with the error "Invalid Option"; delete the line.

The replacement for the trace cache, and a far more powerful one, is mmb2csub (see "Speeding Up the Busiest Routines" below). Where the trace cache replayed individual statements, and gained a few percent when it gained anything, mmb2csub compiles a whole SUB or FUNCTION to machine code, typically 10 to 35 times faster.

Build Availability

OPTION PROFILING is available in every build. The per-subroutine times in its report need call-frame tracking, which is compiled into these builds only:

Build	MCU	Command and call counts	Per-SUB times
PICO	RP2040	Yes	Yes
PICOUSB	RP2040	Yes	-
PICOMIN	RP2040	Yes	-
VGA	RP2040	Yes	-
VGAUSB	RP2040	Yes	-
WEB	RP2040	Yes	-
PICORP2350	RP2350	Yes	Yes
PICOUSBRP2350	RP2350	Yes	Yes
VGARP2350	RP2350	Yes	Yes
VGAUSBRP2350	RP2350	Yes	Yes
WEBRP2350	RP2350	Yes	-
PICOBTRP2350, PICOBTHRP2350	RP2350	Yes	Yes
HDMI, HDMIUSB, HDMIWEB	RP2350	Yes	Yes

The optimisations described in the rest of this document are in every build.

OPTION PROFILING and Built-in Optimisations

PicoMite MMBasic - Performance Optimisation Guide

OPTION PROFILING

Syntax

```
OPTION PROFILING ON
OPTION PROFILING OFF
```

Description

OPTION PROFILING ON allocates the per-command and per-subroutine counters. From then on every statement executed is counted against its command, and every call against its subroutine or function. OPTION PROFILING OFF frees the counters and stops counting.

When the program ends at an END statement, a [PERF] report is printed. It gives the elapsed time, the number of statements executed, the number of variable lookups, the 20 most executed commands and the 20 subroutines and functions with the most calls. On the builds with per-SUB times it also lists the 20 subroutines and functions with the most exclusive (self) time, with their inclusive time and time per call.

Profiling costs nothing when it is off: no code runs and no memory is allocated. The report is printed only by END, so a program to be profiled must finish by reaching an END statement.

Example

```
Option Profiling On
' ... run your program ...
End
```

The report looks like this (per-SUB times build):

```
[PERF] elapsed=16222823 us  statements=714912  findvar=2843723 (locals=2226036 [78%] globals=617687 [21%])  user
[PERF] top commands by dispatch count:
      453246  Let
       83459  Next
       ...
[PERF] top SUBs by exclusive (self) time:
       self_us    incl_us     calls   self_us/call  name
       5572779     7199742       477         11682  moon
       ...
[PERF] top SUBs by call count:
        1455  utc2tdb
       ...
```

Profiling Overhead

OPTION PROFILING ON adds one counter increment per statement and a timestamp per subroutine entry and exit. The counters are allocated once. The run-time cost is typically a few percent on integer-heavy code, so the times in the report are close to, but not the same as, the times of an unprofiled run.

OPTION PROFILING and Built-in Optimisations

PicoMite MMBasic - Performance Optimisation Guide

Speeding Up the Busiest Routines: mmb2csub

In most programs that are too slow, the report shows a few subroutines or functions taking most of the time, and most of that time is the interpreter reading their statements rather than doing the arithmetic in them. No amount of tightening the BASIC changes that. The mmb2csub tool (user-tools/mmb2csub.py) compiles such a routine to machine code and puts it back into your program as a CSUB. The original routine is commented out and the calls to it do not change, because MMBasic calls a CSUB exactly as it calls a SUB.

```
python mmb2csub.py myprogram.bas PlotJulia
```

What to expect:

What the routine does	Speed-up
Loops, array work, integer and float arithmetic	10-35x
Transcendental maths (SIN, LOG), string formatting	3-4x

So the way to a faster program is: profile, find the one or two routines at the top of the self-time list, and convert those. See the separate manual mmb2csub.pdf for what the tool accepts, how to install the compiler it needs, and worked examples.

Optimisations Active in All Builds

The following are compiled into every build and are not controlled by any OPTION.

DO Loop Fast Path

When a DO WHILE, DO UNTIL, LOOP WHILE or LOOP UNTIL condition has the form variable OP number or number OP variable, where the variable is a numeric scalar and OP is one of < > <= >= = <>, the condition is resolved once. The variable is found once and a pointer to its value is kept in the DO stack entry, so each LOOP compares directly instead of running the expression evaluator.

```
Do While i% < 100000      ' fast path: i% compared directly with 100000
  Inc i%
Loop

Do
  x = x + 0.1
Loop Until x >= 4.0       ' fast path, set up at the first LOOP
```

- The comparison is exactly the evaluator's: as integers when both sides are integers, otherwise in floating point. So a fractional limit on an integer variable (Do While i% < 2.5) behaves as it does without the fast path.
- The number is read by the evaluator itself, so decimal, &H, &O and &B literals all work.
- A condition on the DO line is set up when the loop starts; one on the LOOP line at the first LOOP, where it is

first evaluated, so any error is reported where it always was.
- A variable that does not exist yet, an array element, a string, a structure member, a constant or a more complex condition (for example one with AND) simply uses the normal evaluator.
- ERASE of the variable inside the loop switches the fast path off for that loop, which then evaluates the condition normally.

On the PC3 at 378 MHz, 100,000 passes of DO WHILE i% < 100000 : INC i% : LOOP take about 357 ms; on an RP2040 at 315 MHz about 600 ms, against about 1500 ms with the evaluator.

IF / ELSEIF / ELSE / ENDIF Jump Table

Every multi-line IF block in the loaded program (and in the library, if one is loaded) is indexed by PrepareProgram into a sorted table of token-address entries. Each entry records:

- the address of the IF / ELSEIF / ELSE token,
- the address of the next sibling arm (the next ELSEIF, ELSE, or ENDIF),
- the address of the matching ENDIF,
- the source line pointer for error reporting.

At run time, cmd_if and cmd_else perform an O(log N) binary search to jump straight to the next arm or past the ENDIF. Without it they would walk every intervening statement on every false branch, so the gain is largest in deeply nested IF blocks and in IFs that wrap large bodies (typical state machines, menu dispatchers, etc.).

Single-line IF expr THEN cmd [ELSE cmd] is not entered into the table; it is resolved inline from the parsed argument list with no scan at all.

If the table cannot be built (allocation failure, IF nesting deeper than 64, unmatched ELSE/ENDIF) the affected entries are left empty and the run-time falls back to the linear scan, with the same error reporting. Library code and program code occupy separate sorted slices of the same table, so a binary search picks the correct slice based on which memory region the IF token lives in.

The table is rebuilt automatically by RUN, NEW, LOAD, EDIT, and any other operation that calls PrepareProgram. It is freed when the program is cleared.

FOR / NEXT Direct Pointer Stack

cmd_for performs the matching-NEXT scan exactly once per loop entry and stores the resulting program-memory pointer in the forstack slot, alongside a direct pointer to the loop variable's storage. Subsequent iterations therefore:

- reach the loop variable through the saved pointer (no name lookup),
- compare pointers, not strings, between cmdline and the saved nextptr to identify the matching FOR slot when cmd_next runs,
- apply the saved integer or float STEP and TO limit with a single load and compare.

This is what makes a tight For i = 0 To N loop faster than the equivalent Do ... Loop Until with a complex condition. Multi-variable NEXT i, j reuses the same pointer chain to close several loops in one statement without rescanning. The STEP value, TO value, variable type and local level are all captured at FOR entry, so changing the step variable inside the loop has no effect (matching standard BASIC semantics).

g_forindex provides up to MAXFORLOOPS nested levels. Re-entering the same loop variable (e.g. via a GOTO

out of the loop) safely removes the stale stack entry before pushing a new one.

GOTO / GOSUB Line and Label Lookup

Numeric line targets and label targets used by GOTO and GOSUB are resolved by findline / findlabel each time. Labels are stored in a per-program table built by PrepareProgram and found through that table rather than by re-reading the source, and findline walks line headers using the skip byte (below).

commandtbl_decode / Token-Indexed Dispatch

Every command in ProgMemory is stored as a packed CommandToken rather than a string. ExecuteProgram decodes the token with one indexed table load and dispatches through the command table.

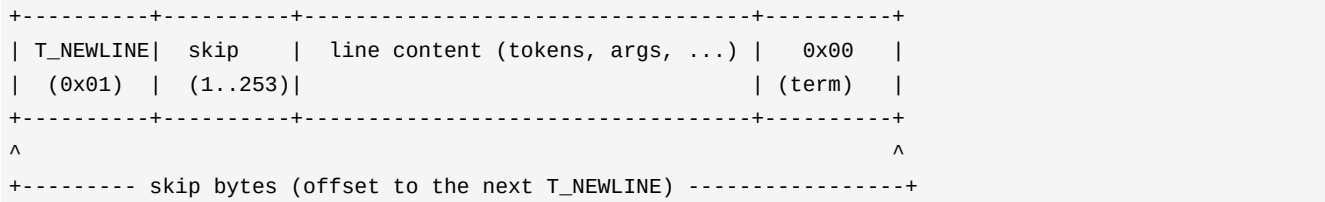
Single-Pass Program Preparation

PrepareProgram walks the loaded program once and in that single pass:

- builds the IF/ELSE jump table described above,
- builds the label table used by GOTO/GOSUB,
- validates SUB/END SUB, FUNCTION/END FUNCTION, and TYPE/END TYPE pairing,
- registers each SUB/FUNCTION in the call-target table for O(1) dispatch.

Tokenised Line Format with Skip Byte

Every program line in ProgMemory (and in the optional library area) starts with a 2-byte header: the T_NEWLINE marker (0x01) followed by a skip byte that records the in-flash length of the line, i.e. the offset in bytes from this T_NEWLINE to the next T_NEWLINE or the end-of-program terminator.



Encoding of the skip byte:

Value	Meaning
3 .. 0xFD (3..253)	Exact byte offset from T_NEWLINE to the next line header
0xFE (T_NEWLINE_SKIP_NONE)	Line too long for one byte (>253): fall back to structural scan
0 and 0xFF	Reserved (would conflict with the 0,0 end-of-program terminator and with flash-erased state)

The skip byte is written by the tokeniser (tokenise() in MMBasic.c) immediately after each line is compacted into tknbuf, by measuring from tknbuf[0] to the first 0,0 pair. Because the in-RAM/in-flash writers stop copying at the first 0,0, the value stored is the same length the writer will commit. The macro T_NEWLINE_HDR (= 2) is the size of the header and is what every p += T_NEWLINE_HDR; site advances.

OPTION PROFILING and Built-in Optimisations

PicoMite MMBasic - Performance Optimisation Guide

A diagnostic verifier (g_verify_line_skip, Commands.c) walks the program and checks that every skip byte either accurately points at the next T_NEWLINE (or end-of-program) or is T_NEWLINE_SKIP_NONE. It is off by default and used during firmware development to validate the tokeniser.

How the skip byte is used

The skip byte turns several O(line length) byte-scans into O(1) jumps. In each case the slow path is kept as a fallback when the skip byte is T_NEWLINE_SKIP_NONE (line too long) or otherwise unusable.

Comment fast path in ExecuteProgram (MMBasic.c). When the interpreter reaches a comment ('), it would normally walk the comment byte by byte looking for the next T_NEWLINE. The fast path instead jumps straight to the next line using the skip byte, so comment-heavy programs spend constant time per comment line.

Line search in findline (MMBasic.c). When GOTO 1234 or RESTORE 5000 needs to find a numeric line number, the scanner peeks p[T_NEWLINE_HDR] for T_LINENBR; if the peek does not match the target, it advances p += skip and skips the entire line in one step instead of scanning each token.

Label hashing in hashlabels (MMBasic.c, RP2350 builds). PrepareProgram builds the label hash by walking every line and inspecting only the first non-newline / non-line-number token. If that token is not T_LABEL, the loop jumps straight to the next line via the skip byte.

PrepareProgram structural scans (label table, IF/ELSE jump table, sub/function registration). The DATA-statement scanners and llist advance past each line header with p += T_NEWLINE_HDR and then walk every colon-separated clause normally. They never use the skip byte to jump over whole lines, so a DATA (or any other statement of interest) after a colon mid-line is always visited.

Per-line copy in flash writers. Because the skip byte is non-zero (range 3..0xFE) and is stored at offset +1 from T_NEWLINE, per-line copy loops that stop at two consecutive zero bytes never see a false terminator inside the header itself.

Tuning Variable Lookup

OPTION LOCAL VARIABLES n

```
OPTION LOCAL VARIABLES n      ' n = 32 .. MAXVARS-32
```

Splits the variable name hash table between local and global slots. The variable lookup that runs for every variable reference probes the local hash first, then the global hash; reducing the number of hash collisions in either domain speeds up every name lookup.

The default split (MAXLOCALVARS) is balanced for a typical program with a few dozen LOCALs per sub and a few hundred globals. Programs with very many globals and few LOCALs benefit from a smaller n; programs with deep sub trees and many LOCALs benefit from a larger n.

The setting must be issued before any DIM or LOCAL statement runs (typically at the very top of the program, before any sub call).

OPTION PROFILING and Built-in Optimisations

PicoMite MMBasic - Performance Optimisation Guide

LIST COLLISIONS

```
List Collisions
```

Diagnostic that reports any variable name groups that hash to the same bucket in the current LOCAL or GLOBAL domain. Use this after a representative run to decide whether OPTION LOCAL VARIABLES n should be retuned, or whether a particular variable should be renamed to break a collision. A program with no reported collisions is already at the lookup-speed floor for that hash configuration.